Progress Report 2

Private and Verifiable Ethereum State Reads

CS6666 | Blockchain and Distributed Systems

Rohan Arora | Roll No. **CS26S022**

1. Progress Since the First Report

Since Report 1, the project has progressed from workload and provider checks to a retained proof corpus, native proof verification, candidate binary records and local CPU PIR retrieval. Helios has been selected as the consensus light client. Its root interface has been tested using synthetic consensus fixtures.

> **Current milestone.** The real proof records can be retrieved through PIR and verified locally under the supplied execution state root. Live consensus authentication of that root and a complete network service remain pending.

2. Real Corpus and Native Proof Verification

A corpus of 32 declared addresses was collected at Sepolia execution block **11919648** for the same WETH9 contract used in Report 1. Every proof request used the exact block hash through EIP-1898. A second provider reproduced the normalised corpus. A separate historical retention failure was preserved without changing the selected block or falling back to another selector.

The Rust libraries `alloy-trie` and `eth_trie` were compared on the retained proofs. Both verified all records under the supplied root and passed 313 rejection checks per candidate. **eth_trie 0.6.1** was selected because it recovers authenticated account and storage values, fitting the proof based manifest without relying on provider supplied account field hints.

Item	Verified local result
Corpus	32 records at one fixed block hash, with the shared WETH account proof
Balance outcomes	24 nonzero inclusions and 8 proven absences under the supplied root
Reusable verifier	Separate account and storage stages, independent key and snapshot binding, byte limits and typed failures
Workspace validation	27 passing Rust tests, including malformed proofs, false values and missing path rejection

The verifier recovers the account storage root before checking the requested storage key. Missing paths, malformed nodes and false values fail explicitly. A zero balance requires a valid absence proof. These results remain conditional on the supplied root, whose consensus provenance is unverified.

Integration decision

The verifier consumes project owned types and performs no RPC or consensus synchronisation. PIR transports the record bytes, while a separate light client will supply the expected root. This keeps proof correctness, selection privacy and root authentication independently testable. The selected library's internal types do not enter the protocol crate.

3. Helios Selection and Root Interface

Helios has been selected for independent execution root authentication. A separate probe verifies a trusted bootstrap and five signed updates from retained synthetic Deneb fixtures. It exports the accepted finalised execution block number, block hash and state root. Ten Rust boundary tests and four source integrity tests cover corrupt updates, incorrect network or checkpoint, stale contexts and provider root substitution.

The probe distinguishes optimistic progress from finality. In one transition, the optimistic head reaches slot 96 while the finalised head remains at slot 72. The adapter continues to export the execution context from slot 72 until a verified finality update advances it. A timeout never forces an optimistic head into finality.

These fixtures use a 32 member test committee and do not authenticate the real WETH corpus. Live Sepolia sync, checkpoint provenance, current fork support and restart recovery remain pending. The choice between core only embedding and reuse of fuller Helios synchronisation machinery is still open.

4. Proof Record Layout and Storage Measurements

A bounded experimental binary codec was built to measure complete storage proof records without JSON hex expansion. Every real row round trips through encoding, padding, decoding and native verification. Oversized rows and malformed frames are rejected explicitly.

Representation	Measured result on the 32 records
Compact JSON	4,599 to 5,545 bytes per record
Binary before padding	2,254 to 2,731 bytes per record
3,072 byte rows	All records fit, with 18.05% of database bytes used as padding
4,096 byte rows	All records fit, with 38.54% padding

Three KiB is the smallest tested capacity that fits this corpus, not a general Ethereum proof bound. At 2,560 bytes, ten records are rejected as oversized. Final geometry must account for larger corpora, future proof sizes and backend alignment rather than discarding proof nodes to meet a row limit.

The common account proof is stored once in public metadata. Storage proofs contain 192 node references but only 142 distinct nodes. Deduplication could remove 35.84% of raw storage proof bytes before private indexing and padding costs. This is a storage observation, not a demonstrated retrieval improvement. Privately obtaining several nodes may cost more than retrieving one complete proof bundle.

5. Local Private Retrieval Experiments

The pinned IKPIR implementation was integrated through an isolated adapter. Direct index PIR and full keyword lookup were tested with SimplePIR and FrodoPIR, at both 3,072 and 4,096 byte record sizes. All eight configurations recovered and verified all 32 records, giving **256 successful local PIR reads**. Source archives, licences, parameters and raw results are retained.

Engine	Retrieval mode	Setup minimum (bytes)	Query + reply (bytes)
SimplePIR	Direct index	15,667,284	12,504
FrodoPIR	Direct index	19,243,072	12,504
SimplePIR	Keyword	125,664,173	98,720
FrodoPIR	Keyword	154,345,093	98,720

The table uses 3,072 byte rows. Query and reply figures include the tested message framing. Setup figures are canonical minimum estimates, not measured network transfers, and exclude the common manifest and index map. The exchange runs within a local process. It does not establish network latency, concurrent throughput or a cryptographic security audit.

Interpretation of the results

Direct indexing has lower setup and response costs in this geometry and is the proposed next baseline. It resolves the logical index locally from a public map. Full keyword lookup returns buckets across multiple segments, increasing the amount retrieved per balance. This comparison concerns the tested table layout and does not establish a universally better PIR engine.

The complete padded generation plus metadata and index map is only 109,390 bytes. The current corpus therefore establishes functional integration, not PIR's practical advantage. Whole generation download remains a reference measurement, but the tiny corpus comparison does not settle the architecture or predict the outcome at meaningful scales.

> **Evidence boundary.** Native proof recovery and PIR retrieval have been composed locally under one supplied root. Live consensus authentication, network transport, private trie node traversal and realistic scale evaluation have not yet been demonstrated.

6. Research Contribution and Evaluation Question

The intended contribution is an evidence based account of **when private, verifiable Ethereum reads are practical**. A working PIR query establishes compatibility, but the stronger result is an operating envelope that connects corpus size, proof representation, client resources, read frequency and snapshot freshness to measured costs.

The main question is: for which workloads does selective private retrieval provide a useful tradeoff compared with downloading and caching the declared dataset? The study must also identify when setup, proof duplication, server work or generation refresh dominates. A larger database is a test condition, not evidence by itself that PIR will be preferable.

7. Planned Scale and Workload Study

The 32 real records will remain the regression corpus. Separate scale tests will increase logical record count and database bytes across multiple orders of magnitude. Concrete sizes will be selected after estimating memory costs, with security parameters held fixed. Synthetic workloads will be labelled separately from larger real Ethereum corpora.

Dimension	Planned comparison and evidence
Representation	Complete proof bundles versus deduplicated trie nodes, including padding, index maps and the number of private retrieval rounds
Client lifecycle	Cold setup, repeated reads on one generation and transitions to a new generation, including hints, public metadata and client storage
System resources	Server preprocessing, query computation, memory and latency distributions, with resource limits and failed sizes reported
Reference paths	Whole generation download counted once when cached, plus ordinary revealing RPC as a correctness and cost reference

Repeated copies of the current records can isolate backend scaling, but cannot establish realistic trie node sharing, proof diversity or update behaviour. Those properties require a declared real corpus. Network measurements will include transport, or will be clearly labelled as a model. Local computation timings will not be presented as complete request latency.

The comparisons will use matched logical queries, proof requirements and snapshot identities. Any private node traversal must hide selection through its access pattern, including retrieval count and stopping behaviour. A smaller database alone is insufficient if it requires more private rounds or reveals part of the requested path.

8. Pending Integration and Next Milestone

- Run the scale and representation study, then review the final record layout and PIR baseline.
- Validate live Helios synchronisation and recovery, then bind a new corpus to an accepted root or establish an authenticated history path for an older block.
- Implement authenticated generation commitments, atomic publication and network serving before demonstrating a complete private and consensus verified read.

The project remains in Phase 1. Verifier and Helios selection are accepted. Final PIR adoption, record geometry, the Helios integration mode and repository licence remain open. No practical crossover or production privacy claim is asserted before the planned measurements and integration checks.

Literature and implementation references

1. **eth_trie 0.6.1.** Value recovery and proof verification interfaces used in the Rust adapter. Source
2. **IKPIR.** Pinned CPU implementation used for direct and keyword retrieval experiments. Source
3. **Helios.** Pinned consensus core used for signed update and finalised root interface tests. Source
4. **SimplePIR and DoublePIR.** Client preprocessing and communication tradeoffs, USENIX Security 2023. Source
5. **EIP-1186.** Native account and storage proof fields used by the collector and verifier. Source